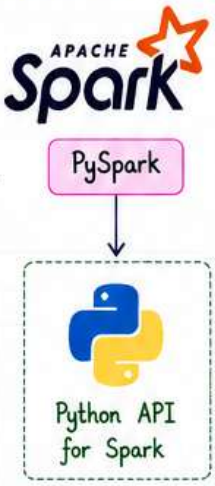
PySpark & Distributed Computing Basics

Suyog Patil
(suyogroww)

1 What is Apache Spark and PySpark?

- Apache Spark is an open source distributed computing engine for big data processing.
- It processes data in memory which makes it much faster than traditional systems.
- PySpark is the Python API for Apache Spark.
- It allows us to write Spark applications using Python instead of Scala or Java.



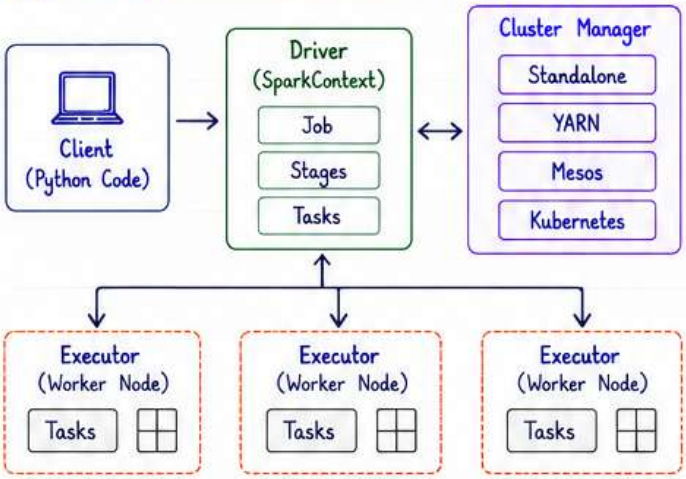
Key Features

- Fast (in-memory processing)
- Supports batch, streaming, ML and more
- Easy to use with Python (PySpark)
- Runs on cluster and handles large data
- Fault tolerant and highly scalable

2 Spark vs Hadoop MapReduce

Feature	Apache Spark	Hadoop MapReduce
Processing Model	In-memory processing	Disk-based processing
Speed	Faster (up to 100x)	Slower
Data Storage	Uses memory (RAM)	Reads/Writes from disk (HDFS)
Iterative Processing	Very efficient	Inefficient
Fault Tolerance	Yes (via lineage)	Yes
Use Cases	Batch, Streaming, ML, SQL, Graph	Batch processing only
Programming Model	Easy (PySpark, Scala, Java, SQL)	Map and Reduce (Complex)

3 PySpark Architecture



4 Driver, Executors & Cluster Manager

Driver

- Runs the main program (your PySpark code).
- Creates SparkContext / SparkSession.
- Breaks the job into stages and tasks.
- Coordinates with executors through the cluster manager.

Executors

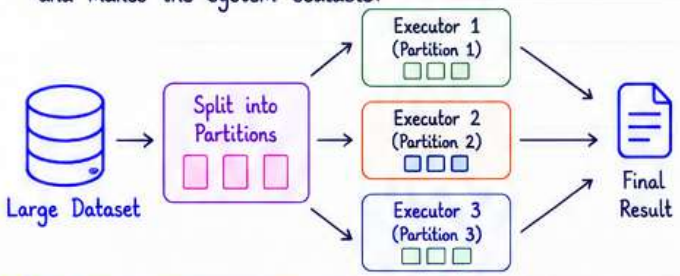
- Run on worker nodes.
- Execute the tasks assigned by the driver.
- Process data in parallel.
- Store data in memory or disk (cache, shuffle).

Cluster Manager

- Manages the resources in the cluster.
- Allocates executors on worker nodes.
- Options: Standalone, YARN, Mesos, Kubernetes.
- Monitors the application and handles failures.

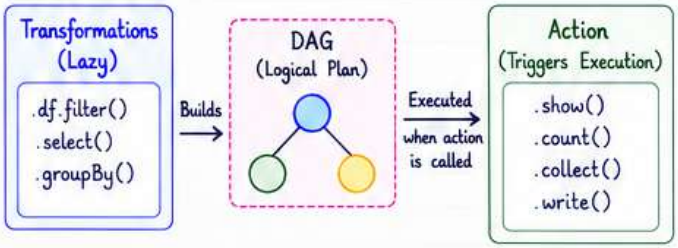
5 Distributed Processing

- Spark splits large data into smaller partitions and processes them in parallel across multiple nodes.
- Each executor works on different partitions of the data at the same time.
- This helps to process huge data faster and makes the system scalable.



6 Lazy Evaluation

- Spark uses lazy evaluation, which means transformations are not executed immediately.
- Instead, Spark builds a logical plan (DAG) of all transformations.
- The actual execution happens only when an action is called.



7 Transformations vs Actions

Transformations

- Used to create a new DataFrame from existing DataFrame.
- They are lazy and not executed immediately.
- Examples: filter(), select(), withColumn(), groupBy(), join(), distinct(), etc.

Actions

- Trigger the actual computation.
- Return a result to the driver program.
- Examples: show(), count(), collect(), take(), write(), save(), etc.

PySpark helps you process large data in a distributed and efficient way using the power of Apache Spark !

SparkSession & DataFrame Essentials

Suyog Patil
(suyogroww)

1 Creating a SparkSession

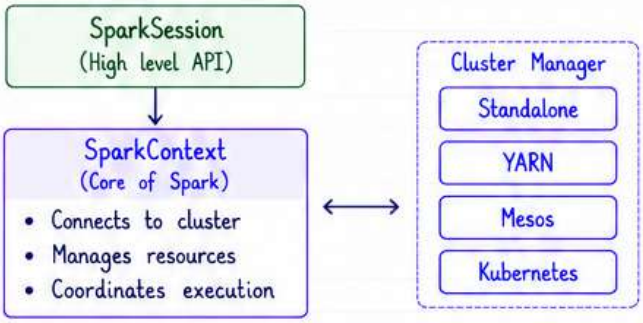
- SparkSession is the entry point to use PySpark.
- It provides a unified interface to work with different Spark features (DataFrame, SQL, Streaming, etc).
- You usually create one SparkSession per application.

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("MySparkApp")
    .master("local[*]")     # use yarn/cluster in prod
    .getOrCreate()
)
```

2 Understanding SparkContext

- SparkContext is the core object behind Spark.
- It connects to the cluster manager and manages resources across the cluster.
- In modern PySpark, you don't create SparkContext directly; it is created automatically when you create a SparkSession.



3 Creating DataFrames

- DataFrame is a distributed collection of data organized in named columns (like a table).
- You can create a DataFrame from many sources:
  - Python objects (list, dictionary)
  - Existing files (CSV, JSON, Parquet)
  - Databases or other data sources

From Python list

```
data = [(1, "Alice", 25),
        (2, "Bob", 30)]
columns = ["id", "name", "age"]
df = spark.createDataFrame(data, columns)
```

Resulting DataFrame

id	name	age
1	Alice	25
2	Bob	30

4 Reading CSV, JSON and Parquet

- Spark can read data from different file formats easily using spark.read.
- These files can be stored in local file system, HDFS, S3 or any supported storage.

Read CSV

```
df_csv = (
  spark.read
  .option("header", True)
  .option("inferSchema", True)
  .csv("data.csv")
)
```

Read JSON

```
df_json = (
  spark.read
  .json("data.json")
)
```

Read Parquet

```
df_parquet = (
  spark.read
  .parquet("data.parquet")
)
```

5 Schema Inference vs Explicit Schema

- Spark can automatically infer the schema from the data or you can define the schema manually.
- Using explicit schema gives better control and can avoid incorrect data types.

Schema Inference

```
df = (
  spark.read
  .option("header", True)
  .option("inferSchema", True)
  .csv("data.csv")
)
```

- ✓ Easy to use
- ✓ Let Spark detect data types
- ✓ Can be slower and sometimes detects wrong data types

Explicit Schema

```
from pyspark.sql.types import *
schema = StructType([
  StructField("id", IntegerType(), True),
  StructField("name", StringType(), True),
  StructField("age", IntegerType(), True)
])
df = spark.read.schema(schema)
        .csv("data.csv")
```

- ✓ Full control over data types
- ✓ More reliable for production
- ✓ Better performance

6 printSchema(), show() and describe()

- These methods help you quickly explore the structure and content of a DataFrame.

printSchema() → Shows the schema with column names and types.

```
df.printSchema()
```

```
root
 |-- id: integer (nullable = true)
 |-- name: string (nullable = true)
 |-- age: integer (nullable = true)
```

show() → Displays the top rows of the DataFrame.

```
df.show(5)
```

```
+---+-----+---+
| id| name|age|
+---+-----+---+
|  1|Alice| 25|
|  2|  Bob| 30|
+---+-----+---+
```

describe() → Shows basic statistics for numeric and string columns.

```
df.describe().show()
```

```
+-------+---+-----+-----+
|summary| id|  age| name|
+-------+---+-----+-----+
|  count|  2|    2|    2|
|   mean|1.5| 27.5| null|
| stddev|0.7|  3.5| null|
|    min|  1|   25|Alice|
|    max|  2|   30|  Bob|
+-------+---+-----+-----+
```

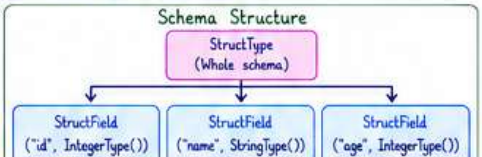
7 StructType and StructField

- StructType is used to define the schema for a DataFrame.
- StructField represents a single column in the schema (name, data type, nullable).

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)
])
```

Schema Structure



Common Data Types

StringType()	-	Text
IntegerType()	-	Integer
DoubleType()	-	Decimal
BooleanType()	-	True/False
TimestampType()	-	Date & time

SparkSession and DataFrame APIs make it easy to work with large datasets using a simple and powerful interface !

Data Manipulation with PySpark

Suyog Patil
(suyogroww)

1 select() and filter()

- select() is used to choose specific columns.
- filter() is used to filter rows based on a condition.

```
from pyspark.sql.functions import col
df.select("id", "name", "age").show()
df.filter(col("age") > 25).show()
```

select()

id	name	age
1	Alice	25
2	Bob	30

filter(age > 25)

id	name	age
2	Bob	30
3	Charlie	28

2 withColumn() and withColumnRenamed()

- withColumn() adds a new column or modifies an existing column.
- withColumnRenamed() renames an existing column.

```
from pyspark.sql.functions import col
df1 = df.withColumn("age_plus_5", col("age") + 5)
df2 = df1.withColumnRenamed("name", "full_name")
df2.show()
```

Result

id	full_name	age	age_plus_5
1	Alice	25	30
2	Bob	30	35
3	Charlie	28	33

Use Cases

- ✓ Create new features
- ✓ Modify existing columns
- ✓ Keep original data unchanged
- ✓ Improve readability with meaningful names

3 drop() and distinct()

- drop() is used to remove columns.
- distinct() is used to remove duplicate rows.

```
# Drop a column
df.drop("age").show()
# Get distinct rows
df.select("name").distinct().show()
```

drop("age")

id	name
1	Alice
2	Bob
3	Charlie

distinct()

name
Alice
Bob
Charlie

4 when() and otherwise()

- Used to create conditional columns (similar to CASE WHEN in SQL).

```
from pyspark.sql.functions import when, col
df = df.withColumn(
    "age_group",
    when(col("age") < 25, "Young")
     .when(col("age") <= 30, "Adult")
      .otherwise("Senior")
)
df.show()
```

Result

id	name	age	age_group
1	Alice	25	Adult
2	Bob	30	Adult
3	Charlie	28	Adult
4	David	40	Senior

5 cast() and Type Conversions

- cast() is used to convert a column to a different data type.
- Helpful when working with strings, numbers, dates etc.

```
from pyspark.sql.functions import col
df1 = df.withColumn("age_str", col("age").cast("string"))
df2 = df1.withColumn("age_int", col("age_str").cast("int"))
df2.show()
```

Result

id	name	age_str	age_int
1	Alice	25	25
2	Bob	30	30
3	Charlie	28	28

Common Conversions

- cast("string") - String
- cast("int") - Integer
- cast("double") - Decimal
- cast("boolean") - Boolean
- cast("date") - Date
- cast("timestamp") - Timestamp

6 Handling Nulls with fillna() and dropna()

- fillna() replaces null values with a given value.
- dropna() removes rows (or columns) containing null values.

fillna()

```
from pyspark.sql.functions import lit
df1 = df.fillna({
    "age": 0,
    "name": "Unknown"
})
df1.show()
```

Result

id	name	age
1	Alice	25
2	Unknown	0
3	Charlie	28

dropna()

```
# Drop rows with any null values
df2 = df.dropna()
df2.show()

# Drop rows based on specific column
df3 = df.dropna(subset=["age"])
df3.show()
```

Result

id	name	age
1	Alice	25
3	Charlie	28

Parameters

- df.fillna(value) → Replace all nulls with a single value
- df.fillna({"col1": value1, "col2": value2}) → Replace per column
- df.dropna(subset=["col"]) → Drop nulls based on specific column
- df.dropna(how="all") → Drop rows where all columns are null
- df.dropna(how="any") → Drop rows where any column is null

7 String and Date Functions

String Functions

- PySpark provides many built-in functions to work with string data.

```
from pyspark.sql.functions import col, upper, lower, trim, concat, substring, length
df.select(
    col("name"),
    upper("name").alias("name_upper"),
    lower("name").alias("name_lower"),
    trim("name").alias("name_trim"),
    concat(col("name"), lit(" - "), col("age")).alias("name_age"),
    substring("name", 1, 3).alias("name_short"),
    length("name").alias("name_length")
).show()
```

Result

name	name_upper	name_lower	name_trim	name_age	name_short	name_length
Alice	ALICE	alice	Alice	Alice - 25	Ali	5
Bob	BOB	bob	Bob	Bob - 30	Bob	3

Date Functions

- PySpark provides functions to work with date and timestamp data.

```
from pyspark.sql.functions import col,current_date,to_date,to_timestamp,
    date_format, year, month, dayofmonth, datediff
df.select(
    col("order_date"),
    to_date("order_date").alias("date"),
    year("order_date").alias("year"),
    month("order_date").alias("month"),
    dayofmonth("order_date").alias("day"),
    date_format("order_date", "yyyy-MM-dd").alias("formatted_date"),
    datediff(current_date(), col("order_date").alias("days_diff")
).show()
```

Common Date Functions

- current_date()
- to_date()
- to_timestamp()
- date_format()
- year(), month(), dayofmonth()
- datediff(), add_months()

Result

order_date	date	year	month	day	formatted_date	days_diff
2024-01-15	2024-01-15	2024	1	15	2024-01-15	120
2024-02-20	2024-02-20	2024	2	20	2024-02-20	84

Aggregations, Joins & Window Functions

Suyog Patil (suyogroww)

1 groupBy() and agg()

- groupBy() is used to group data based on one or more columns.
- agg() is used to apply aggregate functions on grouped data.

```
from pyspark.sql.functions import col, sum, avg, count
df.groupBy("department").agg(
      count("id").alias("total_count"),
      sum("salary").alias("total_salary"),
      avg("salary").alias("avg_salary")
).show()
```

Result

department	total_count	total_salary	avg_salary
Sales	3	180000	60000
IT	2	160000	80000
HR	2	120000	60000

2 count(), sum(), avg(), min() and max()

- These are common aggregate functions used to get summary statistics.
- They can be used with groupBy() or on the entire DataFrame.

```
from pyspark.sql.functions import
count, sum, avg, max
df.agg(
   count("id").alias("total_count"),
   sum("salary").alias("total_salary"),
   avg("salary").alias("avg_salary"),
   min("salary").alias("min_salary"),
   max("salary").alias("max_salary")
).show()
```

Common Aggregate Functions

- count() - Count of rows
- sum() - Sum of values
- avg() - Average value
- min() - Minimum value
- max() - Maximum value

3 Joins in PySpark

- Joins are used to combine data from two DataFrames based on a common column.
- Types: Inner, Left, Right and Full joins.

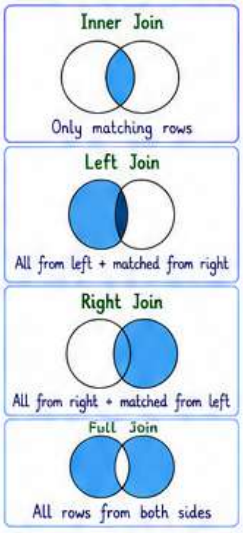
df1 (Employees)

id	name	dept_id
1	Alice	101
2	Bob	102
3	Charlie	103

df2 (Departments)

dept_id	dept_name
101	Sales
102	IT
104	HR

```
# Inner Join
df1.join(df2, "dept_id", "inner").show()
# Left Join
df1.join(df2, "dept_id", "left").show()
# Right Join
df1.join(df2, "dept_id", "right").show()
# Full Join
df1.join(df2, "dept_id", "full").show()
```



4 Broadcast Joins

- Broadcast join is used when one DataFrame is small.
- The small DataFrame is sent to all worker nodes to avoid shuffling large data.
- Useful for improving performance.

```
from pyspark.sql.functions import broadcast
# Broadcast the small DataFrame (df2)
df_joined = df1.join(broadcast(df2), "dept_id", "inner")
df_joined.show()
```

When to use?

- ✓ One of the DataFrames is small (e.g. < 100 MB).
- ✓ Reduces shuffle and network I/O.
- ✓ Improves join performance.
- ✓ Common in star schema joins (fact + dimension).

5 Window Functions

- Window functions perform calculations across a set of rows related to the current row.
- Useful for ranking, running totals, moving averages, etc.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, dense_rank, lag, lead, col

window_spec = Window.partitionBy("department").orderBy("salary")
df.withColumn("row_num", row_number().over(window_spec)).show()
```

Key Components

- partitionBy() - divides data into groups (like SQL PARTITION BY)
- orderBy() - defines the order within each partition
- over() - applies the window specification to a function

Window Specification

```
window_spec = Window.partitionBy("department").orderBy("salary")
```

You can use multiple columns in partitionBy() and orderBy() as well.

6 row_number(), rank() and dense_rank()

- These functions are used to assign a rank or sequence number to rows within a window partition.

id	department	salary
1	Sales	50000
2	Sales	60000
3	Sales	60000
4	Sales	70000
5	IT	45000
6	IT	45000
7	IT	80000

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number,
      rank, dense_rank
window_spec = Window.partitionBy("department").
      orderBy("salary")
df.withColumn("row_number",
      .row_number().over(window_spec)) \
      .withColumn("rank",
      .rank().over(window_spec)) \
      .withColumn("dense_rank",
      .dense_rank().over(window_spec)) \
   .show()
```

Result

id	department	salary	row_number	rank	dense_rank
1	Sales	50000	1	1	1
2	Sales	60000	2	2	2
3	Sales	60000	3	2	2
4	Sales	70000	4	4	3
5	IT	45000	1	1	1
6	IT	45000	2	1	1
7	IT	80000	3	3	2

7 lag() and lead()

- lag() gives the value from a previous row.
- lead() gives the value from a next row.
- Useful for comparing current row with previous/next row.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag, lead
window_spec = Window.partitionBy("department").orderBy("salary")
df.withColumn("prev_salary", lag("salary", 1).over(window_spec)) \
   .withColumn("next_salary", lead("salary", 1).over(window_spec)) \
   .show()
```

Result

id	department	salary	prev_salary	next_salary
1	Sales	50000	null	60000
2	Sales	60000	50000	60000
3	Sales	60000	60000	70000
4	Sales	70000	60000	null
5	IT	45000	null	45000
6	IT	45000	45000	80000
7	IT	80000	45000	null

Use Cases

- ✓ Find previous/next day's value
- ✓ Calculate differences
- ✓ Detect changes
- ✓ Time series analysis
- ✓ Find first/last events

RDDs, Partitions & Data Distribution

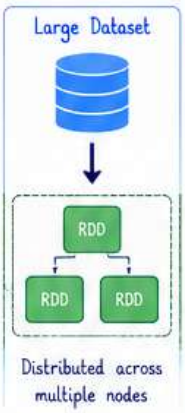
Suyog Patil
(suyogroww)

1 What are RDDs?

- RDD (Resilient Distributed Dataset) is the fundamental data structure in Spark.
- It is a distributed collection of data across a cluster.
- RDDs are immutable (read-only) and fault tolerant.
- Support parallel processing and can be created from various sources (files, collections, etc).

```
# Create an RDD from a Python list
rdd = sc.parallelize([1, 2, 3, 4, 5])
print(rdd.collect())

# Create an RDD from a file
rdd2 = sc.textFile("data.txt")
print(rdd2.collect())
```



2 RDD vs DataFrame vs Dataset

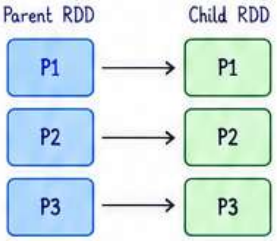
Feature	RDD	DataFrame	Dataset
Data Model	Collection of objects (any type)	Structured data with named columns	Typed, structured data
Type Safety	No	No (untyped)	Yes (strongly typed)
API	Low-level (Functional)	High-level (SQL like)	High-level (Typed, Object oriented)
Optimizations	No	Yes (Catalyst Optimizer)	Yes (Catalyst Optimizer)
Performance	Slower	Faster	Faster
Use Cases	Low-level transformations, custom logic	General data processing	Type-safe processing (in Scala/Java)

3 Narrow vs Wide Transformations

- **Narrow transformations**: each partition of the parent RDD contributes to at most one partition of the child RDD.
- **Wide transformations**: data from multiple partitions is shuffled across partitions.

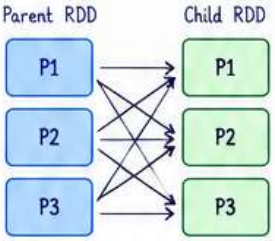
Narrow Transformations

- No data shuffle
- Each partition → one partition
- Examples: map, filter, flatMap



Wide Transformations

- Requires data shuffle
- Data from multiple partitions is combined
- Examples: groupByKey, reduceByKey, join, distinct



4 repartition() vs coalesce()

- repartition(): increases or decreases the number of partitions using a full shuffle (expensive but balanced).
- coalesce(): decreases the number of partitions without a full shuffle (faster, used to reduce partitions).

repartition()

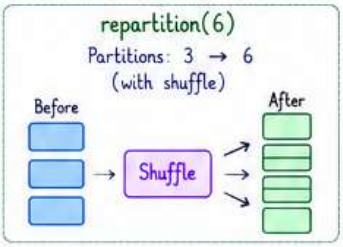
- Can increase or decrease partitions.
- Uses full shuffle.
- Data is evenly distributed.

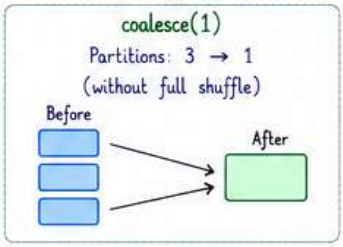
```
rdd2 = rdd.repartition(10)
print(rdd2.getNumPartitions())
```

coalesce()

- Can only decrease partitions.
- No full shuffle (faster).
- Useful after filters/aggregations to reduce partitions.

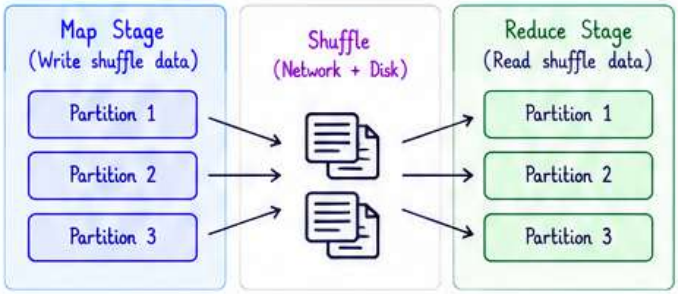
```
rdd2 = rdd.coalesce(5)
print(rdd2.getNumPartitions())
```





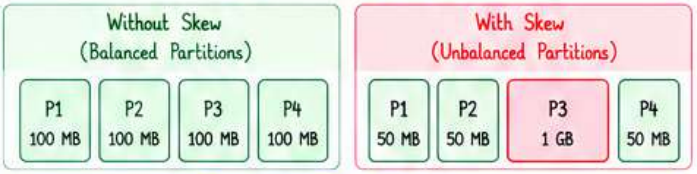
5 Shuffle Operations

- Shuffle is the process of redistributing data across the cluster.
- It happens during wide transformations (e.g., groupByKey, join).
- Data is written to disk, transferred across the network, and read by the next stage.



6 Data Skew Basics

- Data skew happens when some partitions are much larger than others.
- It causes slow performance because some tasks take much longer.
- Common in joins, aggregations and groupBy operations.



Ways to Handle Data Skew

- ✓ Use salting for skewed keys
- ✓ Filter skewed data separately
- ✓ Increase number of partitions
- ✓ Use broadcast joins (for small tables)
- ✓ Repartition by a better key
- ✓ Use AQE (Adaptive Query Execution)

7 Partition Sizing and Parallelism

- Partition size affects performance, memory usage and parallelism.
- Too few partitions → low parallelism, underutilized cluster.
- Too many partitions → high overhead and small tasks.
- Ideal partition size: 100 MB – 1 GB (depends on data and cluster).

Factors to Consider

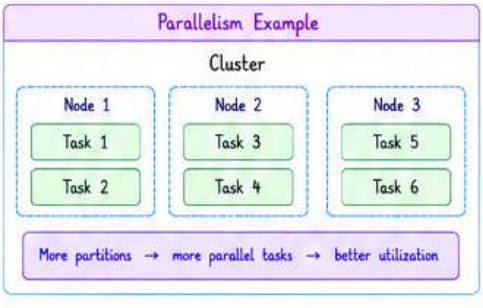
- ✓ Size of input data
- ✓ Number of executors and cores
- ✓ Type of transformation (narrow vs wide)
- ✓ Cluster resources (memory and CPU)

Example

```
# Check number of partitions
print(rdd.getNumPartitions())

# Repartition for better parallelism
rdd2 = rdd.repartition(200)

# Coalesce after aggregations
rdd3 = rdd2.coalesce(10)
```



PySpark Performance Optimization

Suyog Patil
(suyogroww)

1 Catalyst Optimizer

- Catalyst is Spark's query optimizer.
- It automatically optimizes the logical plan of your query using rule-based and cost-based optimization.
- It applies various optimization rules like predicate pushdown, column pruning, constant folding, etc.

Key Optimizations

- ✓ Predicate pushdown
- ✓ Column pruning
- ✓ Constant folding
- ✓ Join reordering
- ✓ Eliminate unnecessary computations

Catalyst Optimization Flow

Unresolved Logical Plan (from SQL / DataFrame)
↓
Analysis (Resolve attributes)
↓
Logical Plan (Optimized)
↓
Physical Plan (Execution strategy)

2 Tungsten Execution Concepts

- Tungsten is Spark's optimized execution engine.
- It uses efficient in-memory data representation and code generation to improve performance.
- It reduces CPU and memory overhead using binary format and off-heap memory.

Tungsten Features

- ✓ Binary data format
- ✓ Off-heap memory
- ✓ Whole-stage code generation
- ✓ Cache-friendly memory layout
- ✓ Reduced serialization overhead

Tungsten Architecture

Optimized Memory Management (Off-heap) → Binary Data Format (Compact) → Whole-stage Code Generation (Faster Execution)

3 Adaptive Query Execution (AQE)

- AQE optimizes the query plan at runtime based on actual data.
- It can automatically change join strategies, coalesce partitions and handle data skew.
- It is enabled by default in recent Spark versions.

```
# Enable Adaptive Query Execution
spark.conf.set("spark.sql.adaptive.enabled", "true")
# Optional configurations
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Benefits

- ✓ Automatically optimizes join strategy
- ✓ Handles data skew
- ✓ Coalesces shuffle partitions
- ✓ Improves query performance without manual tuning

4 Broadcast Joins and Join Strategies

- Broadcast join is used when one of the DataFrames is small.
- Spark broadcast the small table to all worker nodes to avoid shuffle.
- Spark automatically selects the best join strategy based on data size and statistics.

```
from pyspark.sql.functions import broadcast
# Broadcast join (small DataFrame)
df_joined = df_large.join(broadcast(df_small), "id", "inner")
```

Join Strategies	
Broadcast Join	Small table is broadcasted to all nodes
Shuffle Hash Join	Both tables are shuffled and joined using hash
Sort Merge Join	Both tables are sorted and merged (default for large data)
Nested Loop Join	Used for non-equi joins (rarely used)

When to use Broadcast Join?

- ✓ One of the tables is small (typically < 100 MB).
- ✓ Avoids shuffle.
- ✓ Significantly improves performance.
- ✓ You can use broadcast() hint or let AQE handle it automatically.

5 Predicate and Partition Pruning

- Predicate pruning: filters data early to read only required rows.
- Partition pruning: reads only relevant partitions from storage based on filter conditions.

Predicate Pruning

```
df = spark.read.parquet("sales")
    .filter(col("date") >= "2024-01-01")
    .filter(col("region") == "West")
```

Only rows matching conditions are read and processed.

Partition Pruning

```
df = spark.read.parquet("sales")
    .where(col("year") == 2024)
    .where(col("month") == 1)
```

Only required partitions are read from storage.

6 Cache vs Persist

- Caching stores data in memory (default).
- Persist allows you to choose the storage level (memory, disk, etc).
- Use caching for iterative operations to avoid recomputation.

Cache (default: MEMORY_ONLY)

```
df_cached = df.cache()
df_cached.count()  # materialize
```

Use when:

- ✓ Data fits in memory
- ✓ You need quick access multiple times

Persist (custom storage level)

```
from pyspark import StorageLevel
df_persisted = df.persist(
    StorageLevel.MEMORY_AND_DISK
)
df_persisted.count()
```

Use when:

- ✓ Data is large (doesn't fit in memory)
- ✓ You want to store on disk as well

7 explain() and Spark UI

- explain() shows the query execution plan (logical and physical).
- Spark UI helps you analyze job stages, tasks, shuffle operations, and identify performance issues.

Using explain()

```
# Shows formatted execution plan
df.explain()

# Detailed formatted plan
df.explain("formatted")
```

Spark UI (Key Sections)

- ✓ Jobs: stages, tasks, duration
- ✓ Stages: shuffle read/write, task time
- ✓ Storage: cached DataFrames
- ✓ SQL: executed queries and plans
- ✓ Executors: memory and CPU usage

How to Access?

Open Spark UI at: http://<driver-host>:4040

8 Common Performance Bottlenecks

Bottleneck	Cause	Solution
Data Skew	Uneven data distribution across partitions	Use salting, repartition, AQE, or broadcast join
Too Many Partitions	High scheduling overhead	Use coalesce() or tune partition size
Shuffle Overhead	Wide transformations (joins, groupBy)	Reduce shuffle, use broadcast join, optimize join keys
Small Files	Too many small input files	Use file compaction or optimize ingestion
Large DataFrame in Memory	Memory pressure, OOM	Use persist with disk, increase cluster memory
Inefficient UDFs	Python UDFs are slower	Use built-in functions or Pandas UDFs
Non-optimized Joins	Wrong join strategy	Use broadcast join or let AQE optimize

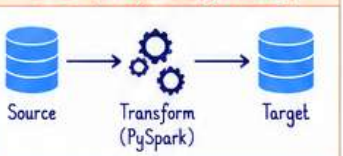
Data Engineering with PySpark

Suyog Patil
(suyogroww)

1 ETL vs ELT

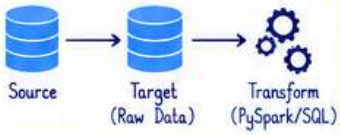
- ETL (Extract, Transform, Load): Transform data before loading into the target system.
- ELT (Extract, Load, Transform): Load raw data first, then transform inside the target system (e.g. Databricks, Snowflake).
- Both approaches are used in data engineering based on use case, data size and tools.

ETL (Transform before Load)



Good for data cleaning and small/medium data

ELT (Transform after Load)



Good for large data and modern data platforms

2 Incremental Data Processing

- Incremental processing loads only new or changed data instead of reprocessing the entire dataset.
- Common methods: timestamp column, high-watermark, or change data capture (CDC).

```
from pyspark.sql.functions import col

# Process only new data based on last processed timestamp
last_timestamp = "2024-01-01 00:00:00"
df_new = df.filter(col("updated_at") > last_timestamp)
df_new.show()
```

Incremental Processing Flow

Read last processed timestamp → Filter new/changed records → Process transformations → Load to target

3 Deduplication and Data Quality Checks

- Deduplication removes duplicate records based on one or more columns.
- Data quality checks help ensure clean, reliable and consistent data.

Deduplication

```
from pyspark.sql.functions
import col

# Remove duplicates based on
a column
df_dedup=df.dropDuplicates(["id"])
df_dedup.show()
```

Data Quality Checks

```
from pyspark.sql.functions import col, count

# Check for null values
df.select([count(when(col(c).isNull(), c))
        .alias(c) for c in df.columns]).show()

# Check for duplicate records
df.groupBy("id").count().filter(col("count") >1)
 .show()
```

4 Reading and Writing Parquet

- Parquet is a columnar storage format, optimized for big data.
- It provides better compression and faster query performance.

Read Parquet

```
df = spark.read.parquet(
    "s3://data/input/sales")
df.printSchema()
df.show()
```

Write Parquet

```
df.write.parquet(
    "s3://data/output/sales",
    mode="overwrite"
)
```

Parquet Advantages

- ✓ Columnar storage format
- ✓ Efficient compression (snappy, gzip)
- ✓ Faster query performance
- ✓ Supports partitioning
- ✓ Widely used in data lakes
- ✓ Optimized for big data processing

5 Append vs Overwrite Modes

- Append mode adds new data to existing data.
- Overwrite mode replaces the existing data.

Append Mode

```
df.write.mode("append") \
    .parquet("s3://data/sales")
```

Overwrite Mode

```
df.write.mode("overwrite") \
    .parquet("s3://data/sales")
```

Mode	Description	When to Use
append	Adds new data to existing data	Incremental loads
overwrite	Replaces existing data	Full refresh loads
ignore	Ignores write if data exists	Avoid accidental overwrite
error (default)	Throws error if data exists	Safer option

6 Partitioned Data Storage

- Partitioning helps organize data into folders based on column values.
- It improves query performance by reading only relevant partitions.

```
df.write \
  .mode("append")
  .partitionBy("year", "month")
  .parquet("s3://data/sales_partitioned")
```

Resulting Folder Structure

```
s3://data/sales_partitioned/
└→ year=2024/
   |  ├── month=01/
   |  ├── month=02/
   |  ├── ...
└→ year=2025/
      |── month=01/
      ├── ...
```

Benefits of Partitioning

- ✓ Faster reads (partition pruning)
- ✓ Better performance for large datasets
- ✓ Easier data management
- ✓ Commonly used columns: date, region, category

7 Handling Bad Records

- Bad records can have incorrect schema, missing values, or invalid formats.
- You can handle them using schema validation, try-catch logic, or by directing them to a separate error path.

Handle Bad Records using PERMISSIVE mode

```
df = spark.read\
    .option("header", True)\
    .option("mode", "PERMISSIVE")\
    .option("columnNameOfCorruptRecord", "_corrupt_record")
    .csv("s3://data/input/sales.csv")

# View bad records
df.filter(col("_corrupt_record").isNotNull()).show()
```

Other Modes

PERMISSIVE
(invalid records go to _corrupt_record column)

DROPMALFORMED
(drops bad records)

FAILFAST
(throws error on bad record)

8 Building Reusable Transformation Functions

- You can create reusable functions to apply common transformations across multiple datasets.
- This makes your code modular, maintainable and easy to test.

Example: Reusable Clean & Transform Function

```
from pyspark.sql.functions import col, trim, when
def clean_sales_data(df):
    return (df
        .withColumn("id", col("id").cast("int"))
        .withColumn("name", trim(col("name")))
        .withColumn("amount", col("amount").cast("double"))
        .filter(col("amount").isNotNull())
        .dropDuplicates(["id"])
```

Usage

```
df_clean = clean_sales_data(df_raw)
df_clean.show()
```

Benefits

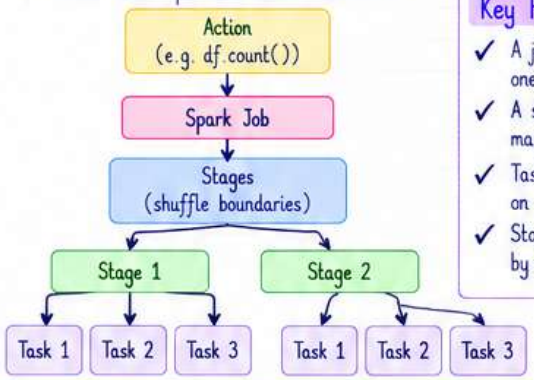
- ✓ Code reusability
- ✓ Consistent transformations
- ✓ Easy to maintain
- ✓ Can be extended for different datasets
- ✓ Helps in building scalable pipelines

PySpark in Production & Interview Essentials

Suyog Patil (suyogroww)

1 Spark Job Execution: Jobs, Stages & Tasks

- When an action is triggered, Spark creates a Job.
- A job is divided into Stages based on shuffle boundaries.
- Each stage is further divided into multiple Tasks.
- Tasks run in parallel across executors.



Key Points

- ✓ A job can have one or more stages
- ✓ A stage can have many tasks
- ✓ Tasks run in parallel on different executors
- ✓ Stages are separated by shuffle operations

2 Local Mode vs Cluster Mode

Local Mode

- Runs on a single machine (for development and testing).
- Master URL: local[*]
- Uses local CPU cores.
- Good for learning and small data.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("LocalMode") \
    .master("local[*]") \
    .getOrCreate()
```

Cluster Mode

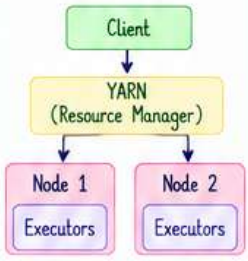
- Runs on a cluster (production).
- Uses a cluster manager like YARN, Kubernetes or a managed platform (EMR, Databricks).
- Distributes work across multiple nodes and executors.

```
spark = SparkSession.builder \
    .appName("ClusterMode") \
    .master("yarn") \
    .getOrCreate()
```

3 YARN, Kubernetes and Managed Spark Platforms

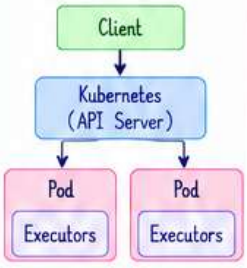
YARN (Hadoop Ecosystem)

- Classic cluster manager for Spark.
- Manages resource allocation and job scheduling.
- Common in on-premises and Hadoop environments.



Kubernetes

- Container-based cluster manager.
- Runs Spark jobs in Docker containers.
- Provides better scalability and resource management.



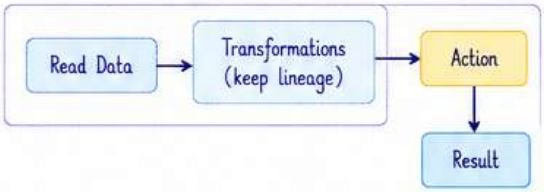
Managed Spark Platforms

- Cloud providers offer managed Spark services.
- Reduces infrastructure management overhead.
- Examples: Databricks, AWS EMR, Google Dataproc, Azure Synapse.

databricks · aws EMR · Google Cloud Dataproc · Azure Synapse

4 Fault Tolerance and Lineage

- Spark automatically handles failures using lineage.
- Lineage keeps track of transformations applied on a DataFrame/RDD.
- If a partition is lost, Spark can recompute it using the lineage graph.



Key Points

- ✓ Data is not lost even if a node fails.
- ✓ Spark recomputes lost partitions using lineage.
- ✓ No need for data replication (like HDFS).
- ✓ Checkpointing can be used for long lineage chains.

5 Logging and Error Handling

- Use logging to track progress, errors and important events.
- Handle exceptions to make jobs more robust.
- Log important information like start time, record counts, etc.

```
import logging

logging.basicConfig(level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

try:
    logger.info("Reading input data")
    df = spark.read.parquet("s3://data/input/")
    logger.info(f"Total records: {df.count()}")
except Exception as e:
    logger.error(f"Error while reading data: {str(e)}")
```

Best Practices

- ✓ Use proper logging levels (INFO, WARNING, ERROR).
- ✓ Log start/end time and record counts.
- ✓ Use try-except blocks.
- ✓ Send alerts for failures (in production).
- ✓ Store logs in a centralized system (e.g. CloudWatch, Datadog).

6 Common PySpark Interview Questions

1. What is the difference between RDD, DataFrame and Dataset?
2. Explain lazy evaluation in Spark.
3. What is a shuffle and when does it happen?
4. What is the difference between coalesce() and repartition()?
5. How does Spark achieve fault tolerance?
6. Explain Catalyst optimizer and Tungsten.
7. What are the different join types in PySpark?
8. How do you handle data skew in Spark?

7 End-to-End ETL Pipeline Example

- Read raw data from a source (e.g. S3).
- Apply transformations (cleaning, deduplication, business logic).
- Write the processed data to a target (e.g. S3/Delta/Parquet).




```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

spark = SparkSession.builder \
    .appName("ETL Pipeline") \
    .getOrCreate()

df = spark.read.parquet("s3://data/raw/")
df_clean = df.dropDuplicates(["id"]) \
    .filter(col("is_valid") == 1)

df_clean.write.mode("overwrite") \
    .partitionBy("date") \
    .parquet("s3://data/processed/")
```

8 Best Practices for Production Workloads

- ✓ Use proper cluster sizing (executors, memory, cores).
- ✓ Partition data correctly.
- ✓ Use broadcast joins for small tables.
- ✓ Enable Adaptive Query Execution (AQE).
- ✓ Cache frequently used data.
- ✓ Avoid unnecessary shuffles.
- ✓ Use data formats like Parquet/Delta.
- ✓ Implement proper logging and monitoring.
- ✓ Handle bad records and data quality checks.
- ✓ Use checkpointing for long jobs.
- ✓ Test jobs in lower environments before production.
- ✓ Set up alerts for job failures.
- ✓ Follow modular and reusable code structure.